

Human Mobility Prediction on 2G/3G Cellular Network via LLM

From raw mobile-network traces to next-cell prediction

Olivier Recher
CVUT Praha

PRESENTATION 1

Documentation & first attempt to train an LLM with our dataset

Olivier Recher

July 3 - July 8

Three ways to train — QLoRA is the fit

full fine-tuning vs LoRA vs QLoRA.

Source: - [How to Train an LLM on Your Own Data](#) (Heavybit, September 2025).

- [Fine-Tuning LLaMA 3.1 on Your Custom Dataset: A Comprehensive Guide](#) (Ujwal Tewari, March 2025).

Full Fine-Tuning

- › Updates 100% of model weights
- › Maximum theoretical performance
- › Massive VRAM consumption

LoRA

- › Adds external low-rank adapters
- › Freezes original model weights
- › Drastically reduces memory needs

QLoRA

- › Quantizes weights to 4-bit precision
- › Trains LoRA adapters on top
- › Ultimate optimization for limited GPUS

The only choice regarding to our limited GPU and VRAM

SELECTED

Two toolchains for the QLoRA run

Standard Hugging Face / PyTorch stack vs the Unsloth framework.

Source: - [How to Train an LLM on Your Own Data](#) (Heavybit, September 2025).

- [Fine-Tuning LLaMA 3.1 on Your Custom Dataset: A Comprehensive Guide](#) (Ujwal Tewari, March 2025).

First attempt



Hugging Face & PyTorch

- › Open-source standard (Transformers, TRL)
- › Large choice of Model
- › Maximum configuration flexibility/hyperparameters

with Google Colab for cloud GPU
but limited free plan per days



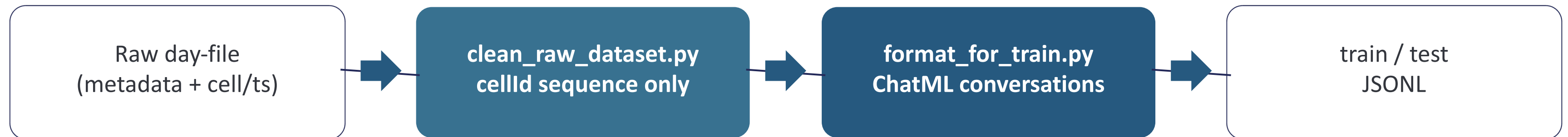
Unsloth Framework

- › Highly optimized library for GPUs
- › Accelerated training speed (up to 2×)
- › Seamless native exports (GGUF, vLLM)

COMING NEXT I will try Unsloth with local training.

Clean DataSet

Raw CSVs carry metadata and hundreds of cell/timestamp columns, we focus on sequence of cell_id to train LLM.



OUTPUT 2014-03-12: 85 946 users -> 64 984 train / 20 962 test (random 75-80% split).

Exemple of ChatML format {"conversations": [{"role": "system", "content": "You are an AI that analyzes user event sequences, DataSet start by User <USER_ID> | <N_RECORDS> events | <CELL_ID> <CELL_ID> ..."}, {"role": "user", "content": "Here is the sequence for this user."}, {"role": "assistant", "content": "User 1468158 | 5 events | BSOSED2 DSOTKB2 DSOTKB2 BSOBUK4 BSOBUK4"}]}

A first signal after only 1.2% of the data

Small Google Colab run: 100 steps on a slice of the training set, evaluated on next cell_id.

NEXT-CELL ACCURACY

16.87%

14 / 83 correct predictions

14 Correct predictions

69 Incorrect predictions

TRAINING RUN (GOOGLE COLAB)

- ~ 30min training time
- model LLM: mistralai/Mistral-7B (8Bit is the max regarding to GPU)
- 100 training steps
- Training loss: from 0.902331 to 0.5825
- Epoch: 0.0122 (~1.2% of the data seen)

NOTE Only ~1.2% of the training data was used, first result and training loss are encouraging signals.

NOTE I also tried using the Llama model, but at this training level, it doesn't even return any cell_id values to me.

What is done, and what comes next

The training pipeline and a first GPU run are in place; coverage is the next lever.

DONE Documentation and Reproducible data pipeline: raw -> cleaned -> ChatML train/test split.

DONE First Result with Mistral: 16.87% next-cell accuracy after 100 steps and with only ~1.2% of the training data.

COMING NEXT Tune training hyperparameter and a longer training session to cover far more than 1.2% of the data and beat 16.87%.

COMING NEXT I will try Unsloth with local training.

PRESENTATION 2

LLM Training with new model and different number of data

Olivier Recher
July 9 - July 14

Model hits the data ceiling, then overfits

Mistral-7B QLoRA · 500 users (31k records) · 8 epochs in ~5.5 h on Colab T4 · best val=41.1% (no gain) — abandoned

Qwen2.5-0.5B QLoRA · 500 users (31k records) · 49 epochs · ~72 min on Colab T4

BEST next cell prediction

64.2%

epoch 9 — saved in best_model

FINAL TEST

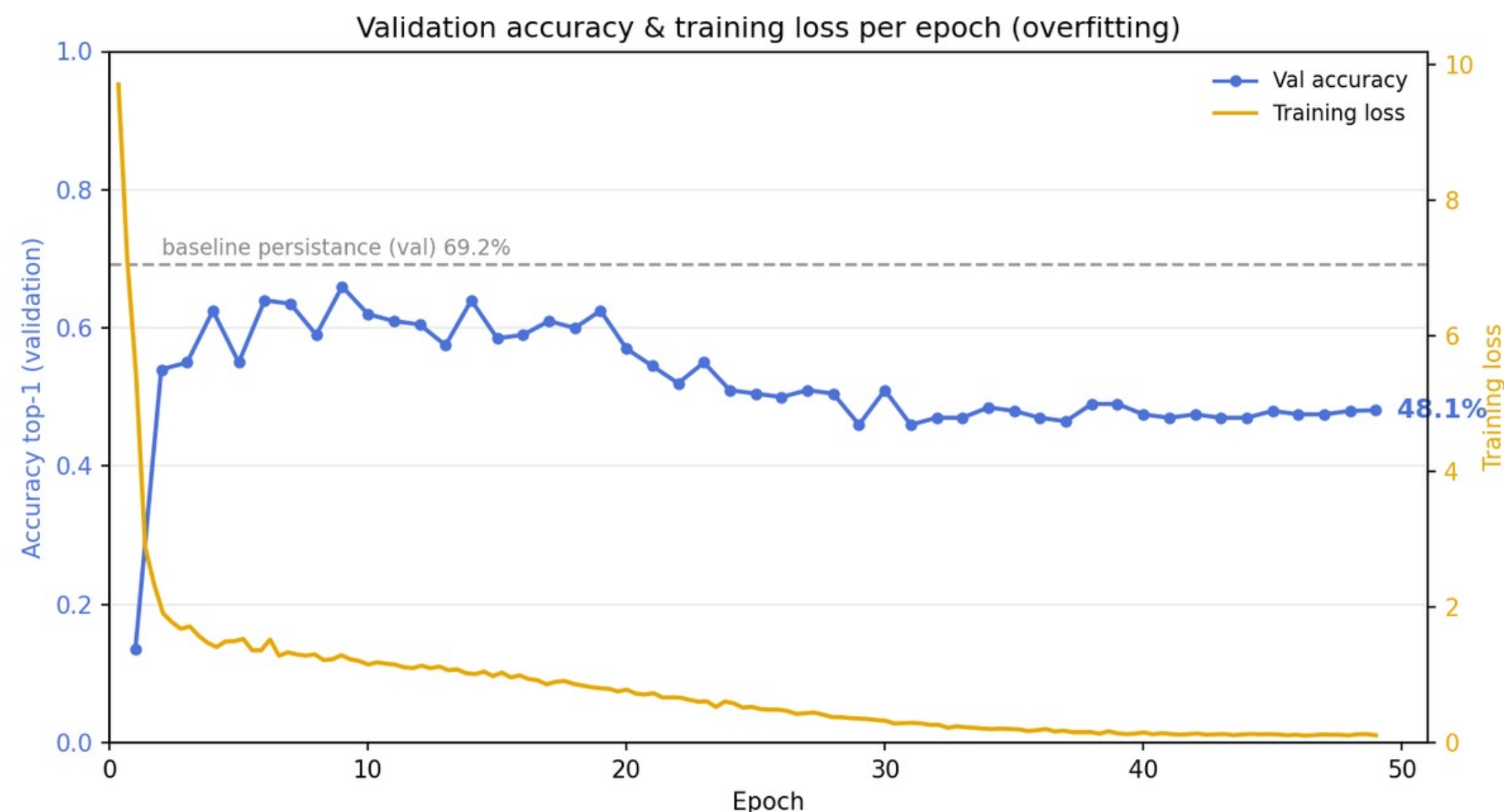
48.1%

last epoch = overfit model

Persistence baseline

≈ 69,2%

percentage of correct answers that are just repeats of the last viewed



- › Train loss **9.7** → **0.11** = overfitting/memorization → validation peaks at **epoch 9**, then drops **-16 %**
- › Under Persistence baseline **69.2%** → the model is worse than not making any predictions at all for the moment
- › **40 wasted epochs** (~50 min GPU) — early-stop patience **40** → **8** ; evaluate the **best** model, not the last

Same model, +16 %: the harness matters

Second run — same Qwen2.5-0.5B QLoRA, same 500 users, engineering fixes only

TEST ACCURACY

48.7% → 64.4%

+15.7 % — evaluate the best model
still below persistente baseline

GPU TIME (COLAB T4)

72 min → 29 min

−60% — early stop at epoch 16

Correct value in TOP-3 ACCURACY

76.2% → 82.7%

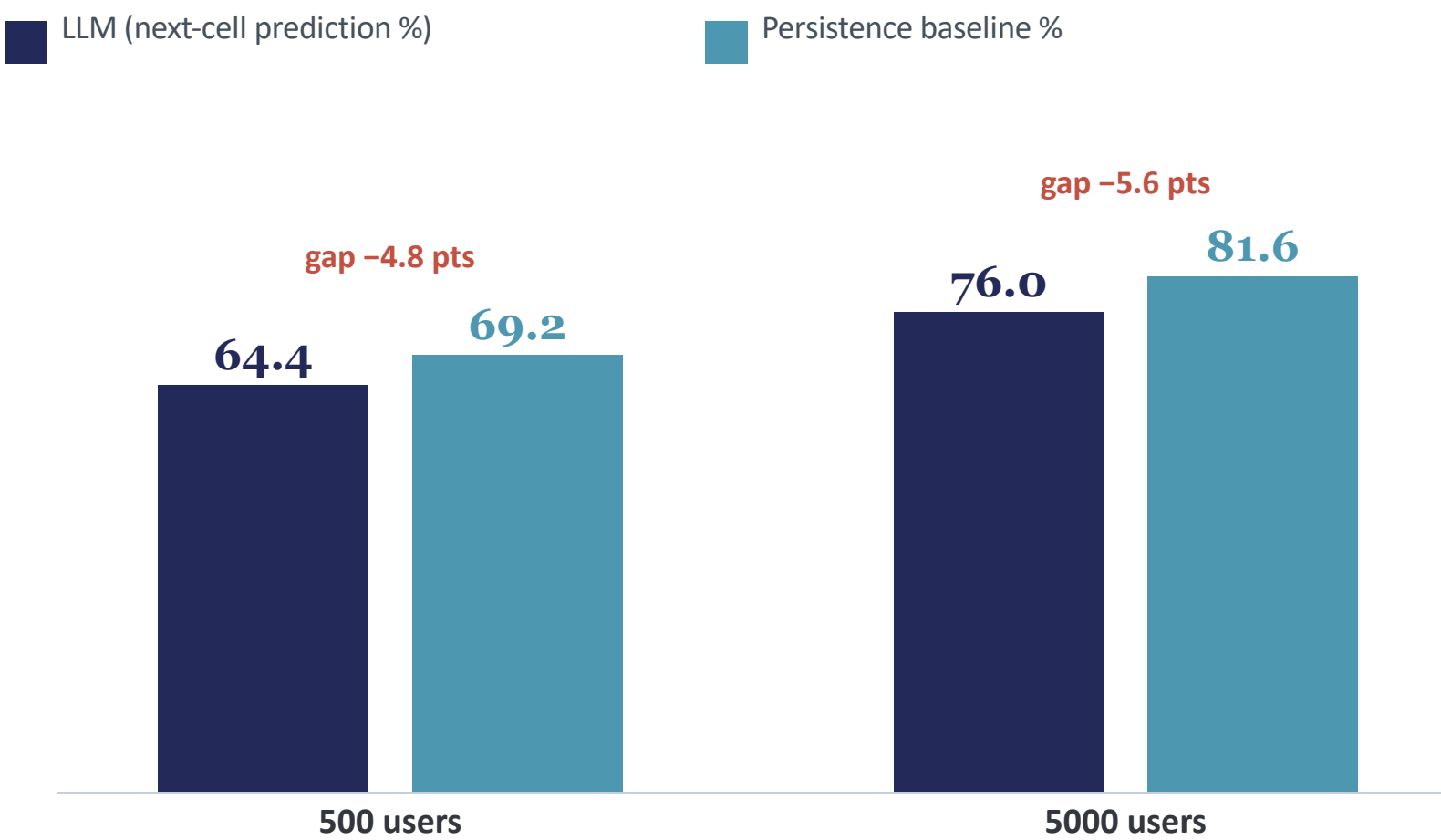
WHAT CHANGED

- › Evaluate the **best checkpoint** | not the last (overfit) one
- › Early stop, **patience 8** | validation peaks at **epoch 8**
- › Anti-overfit: **dropout 0.1 · LR 1.5e-4 · weight decay 0.01**

INFO I had run with 100 users also: model learns but stay below 42.1% + accuracy is very bimodal per-user (10% .. 100%), so 100 users is too few to generalize.

10× the data, still below the baseline

same Qwen2.5-0.5B QLoRA · same 500 users · best checkpoint



The LLM sits ~5 pts under the persistence baseline at both scales — the gap doesn't close. top-3: 82.7% → 88.3%.

TEST TOP-1 ACCURACY

64.4% → 76.0%
rose with the data...

GAP TO PERSISTENCE

-4.8 → -5.6 pts
...but never catches the baseline

TRAINING TIME (COLAB T4)

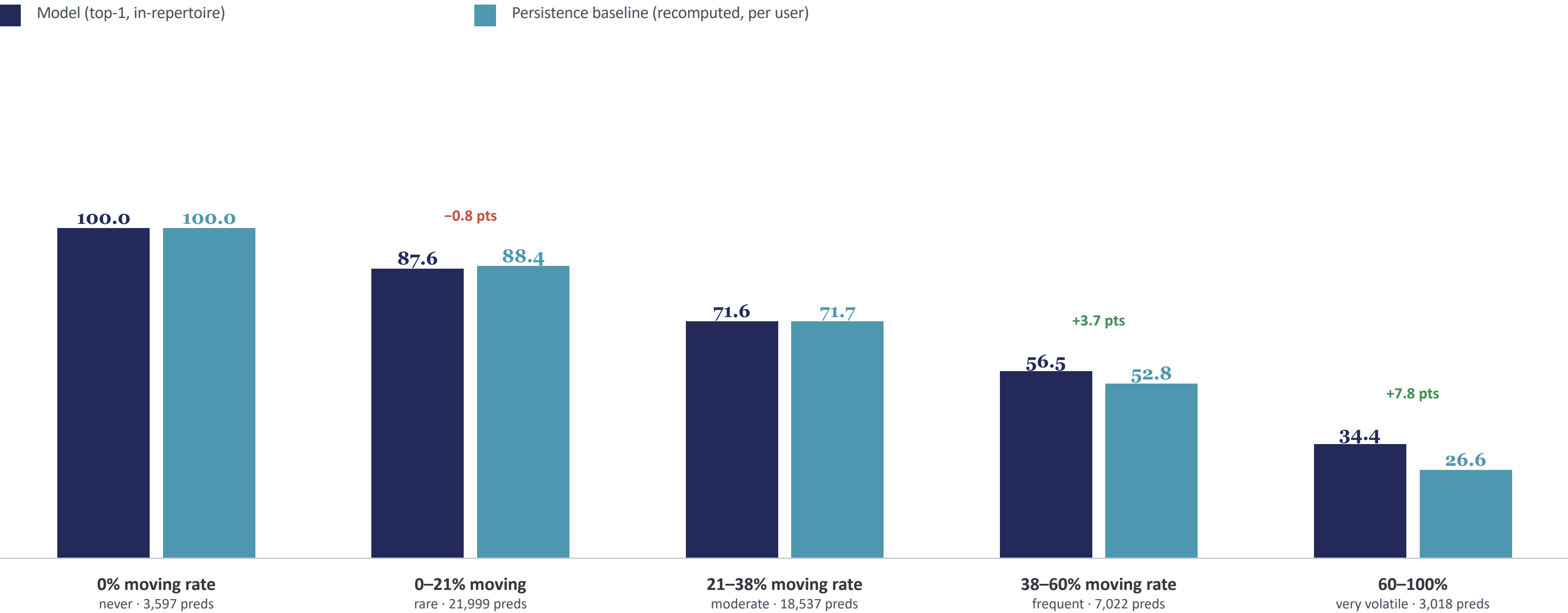
≈29 min → ≈4 h
~8× longer — best model already at epoch ~4

VERDICT Scaling real data 10× lifted top-1 by ~12 %, but the **persistence baseline rose in lockstep**. The model never beats “repeat the last-seen cell” —> it has essentially **learned to copy it**. More data mainly buys training time. The lever now is **reformulating the task** (predict transitions, not stays) or **adding signal** (time-of-day, cell geography). Don't need more users.

user 775719 · prefix BKVSTU1×10
→ BKVSTU1 (100%) · real BKVSTU1 ✓ stay
user 3268099 · prefix ...BKVIV2
→ BKVVIV2 (68%) · real BSOBUK4 ✗ missed move

Accuracy is dictated by how often a user switches cells

1,173 users split into 5 equal-size groups by switch rate | bars show weighted top-1 accuracy per group

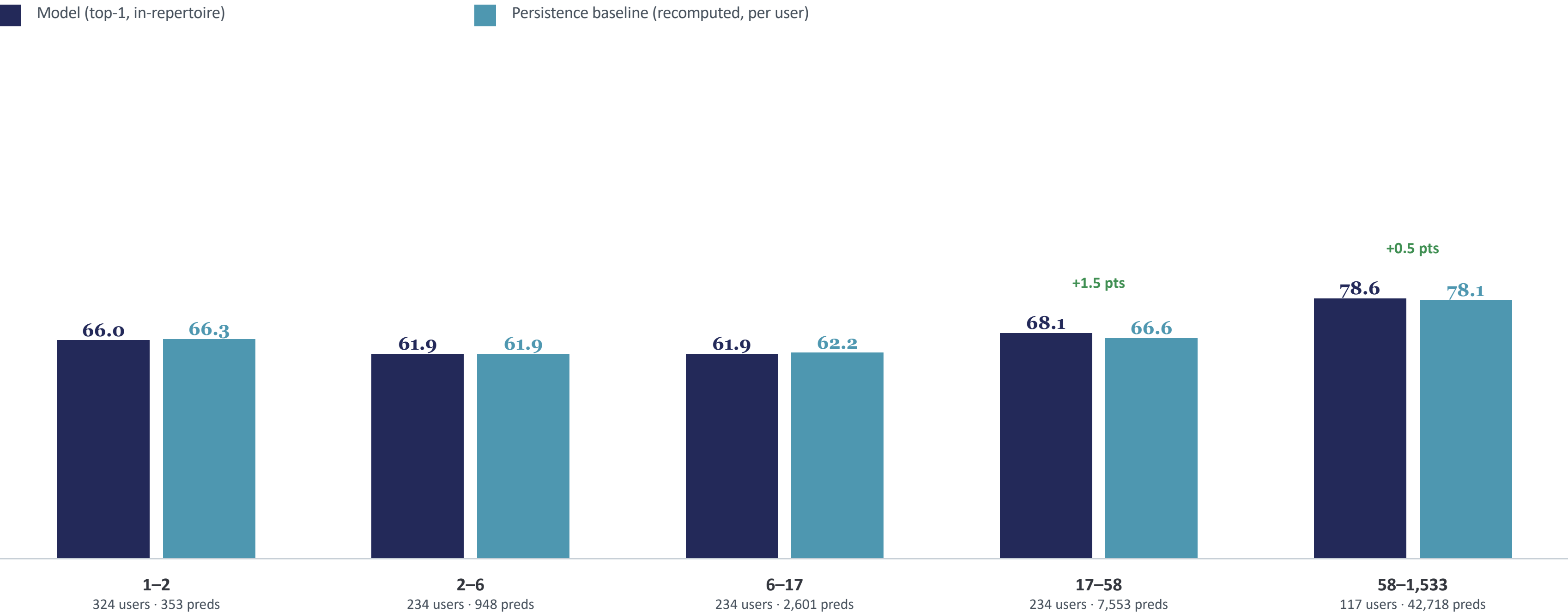


- › Users who **never switch cell** (20% of users) are predicted at 100% —> the task is trivial for them.
- › The most volatile 20% of users plateau at **34.4%** —> a ceiling driven by user behavior, not model capacity.

FINDING 2 — DATA VOLUME IS EXTREMELY CONCENTRATED

10% of users generate 59% of the evaluation signal

Same 1,173 users, now split into 5 equal-size groups by number of predictions per user

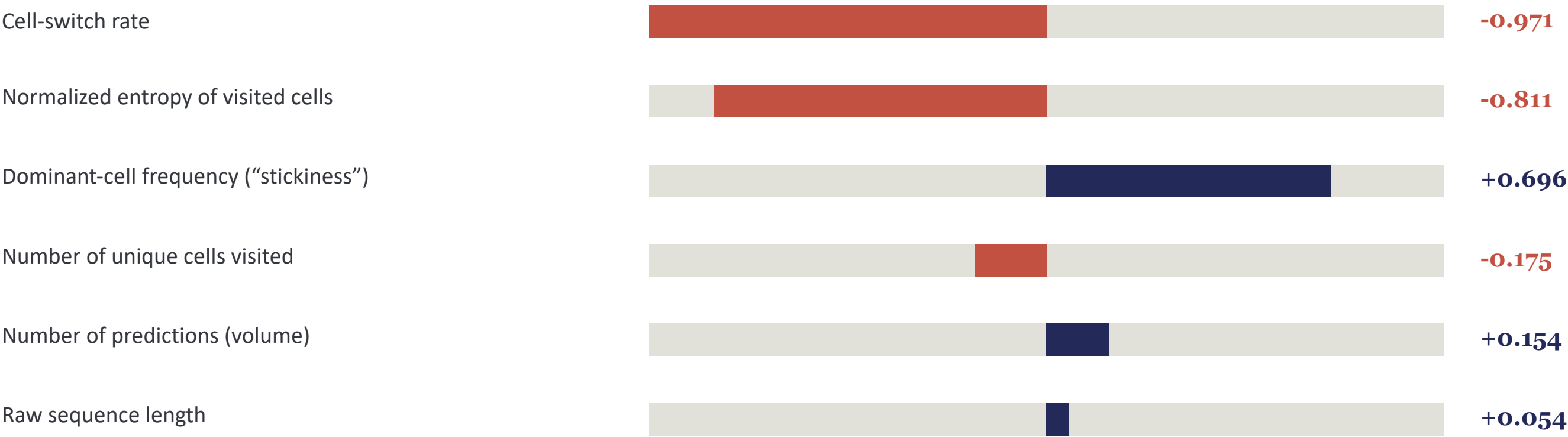


- › Median user has only **11 predictions** but the top 10% most active users (117 people) cover 58.6% of the 54,173 predictions.
- › This is not a volume effect: raw correlation accuracy × volume is weak ($r = 0.15$). High-volume users simply tend to switch cell less often ($r = -0.15$), so the lift here is really Finding 1 in disguise.

Behavior explains the score; the model explains the residual

Pearson correlation between per-user next cell accuracy and each sequence-derived feature

Bar extends right in navy for a positive correlation, left in rust for a negative one, length is proportional to $|r|$.



VERDICT Per-user accuracy is explained almost entirely by **how repetitive the user's mobility already is** (switch rate alone: $r = -0.97$). **The model's real value-add is concentrated in the hardest, most volatile users** —> but that segment is only 5.6% of the evaluation volume, so the lift barely moves the aggregate.

NEXT STEPS

What comes next

Find the next lever.

COMING NEXT I would separate staying data and moving data to train llm on both situation to at least beat persistence baseline.

COMING NEXT Validate the model's real value-add with improving prediction on most volatile users.

COMING NEXT Stay on 500 users dataset and **add signal** (time-of-day, cell geography).

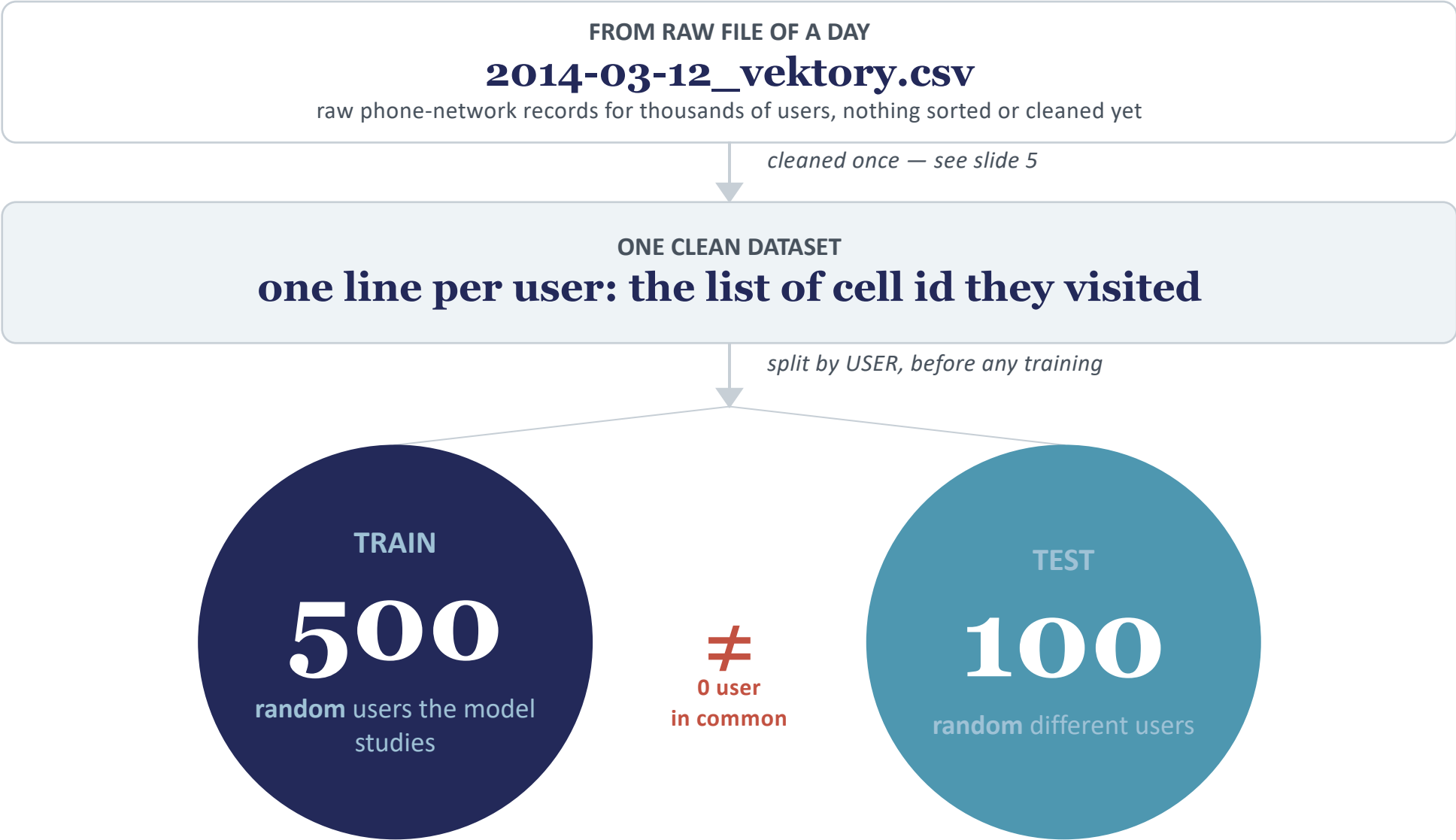
PRESENTATION 3

What exactly do I do to train a model

Olivier Recher
July 14 - July 17

Split data into two groups of users who never meet

Same raw export for everything, cleaned once (slide 5), then split BEFORE any training happens

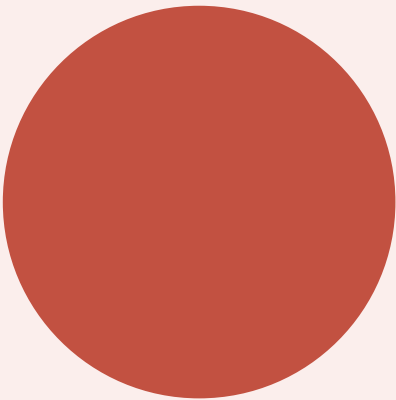


Same file, cut into two groups of people that never overlap: **the model only ever studies the 500** —> it has never seen a single one of the 100 test people before being graded on them.

How do i choose LA model to fit the problem and the GPU

Mistral-7B

Mistral AI · France · Sept 2023



~7 billion parameters

★ famous model for: general reasoning & code

reminder previous test

≈5.5 h on Colab T4

best val = 41.1% — no gain

ABANDONED

Qwen2.5-0.5B

Alibaba Qwen · China · Sept 2024



~0.5 billion parameters (14× smaller)

★ known for: fine-tuning, more malleable

reminder previous test

≈29 min on Colab T4

KEPT

WHY QWEN, NOT MISTRAL

■ Restricted task — doesn't need broad world knowledge

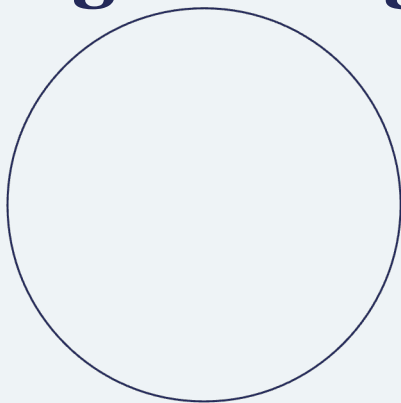
■ 14× smaller → ≈23x faster

■ 5.5h vs. 29min

Same goal, two very different methods on the team

Prompting engineering

Typhaine & Benjamin's approach



0% of the model's weights change

★ what it is: prompt a frozen model

prompting with given different context/dataset

Fine-tuning

My approach



a little part of the model's weights are permanently updated

★ what it is: retrain a part of the model

A real training loop to teach to the model how to predict

THE CORE DIFFERENCE

- Prompting = better instructions, same frozen brain
- Fine-tuning = the brain itself gets reshaped by examples
- Both are valid — just different tools for different jobs

Steps of a training session

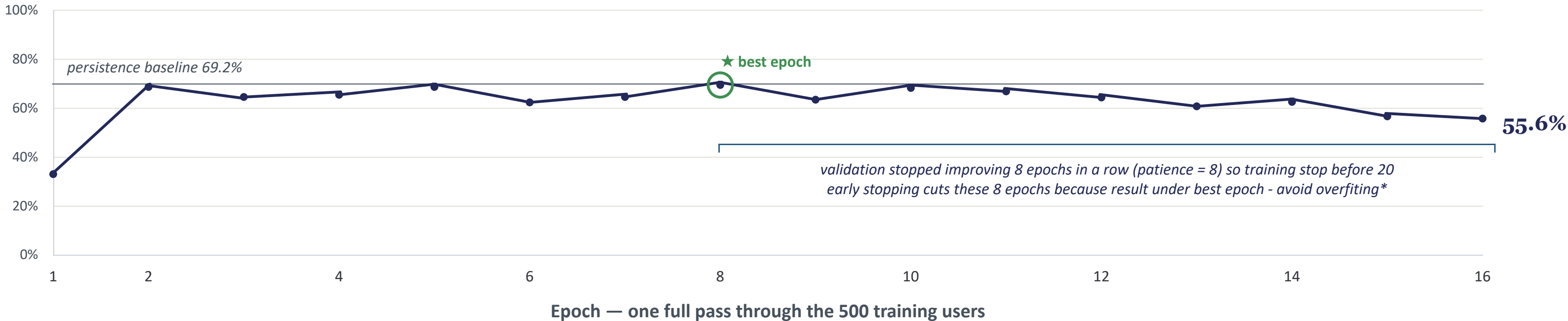
What the training script actually does, in plain terms.



The results, best checkpoint gets evaluated

Qwen2.5-0.5B QLoRA · 500 train users · early-stopped at epoch 16/20 · best model = epoch 8

STEP 1 — an epoch = one full pass through all 500 training users · the chart below tracks accuracy after each one of the 16 passes



i Persistence baseline = predicting the user stays in their last-seen cell, no model at all — the benchmark every accuracy on this slide is measured against.

STEP 2 — grade that checkpoint on the 100 test people it has never seen

THE MODEL SCORES

68.4
%

<
-3.2 pts

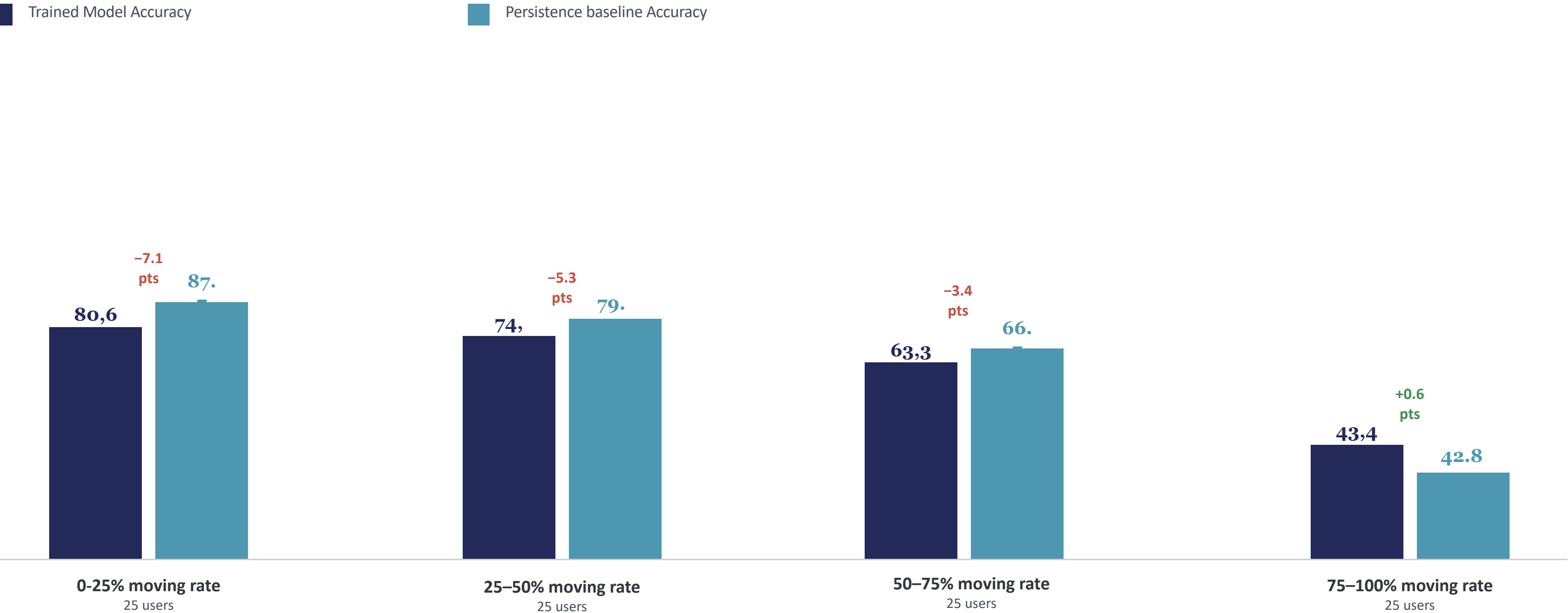
“JUST REPEAT THE LAST CELL” SCORES (Persistence Baseline)

69.2
%

***OVERFITTING, IN ONE SENTENCE** Like a student who memorizes the answer key instead of understanding the lesson —> perfect on questions they've already seen, worse on brand-new ones. So we keep the **best epoch (8)**, not the last one (16).

Accuracy is dictated by how often a user switches cells

The 100 users from dataset_test split into 4 equal-size groups by switch rate / bars show weighted prediction next cell accuracy per group

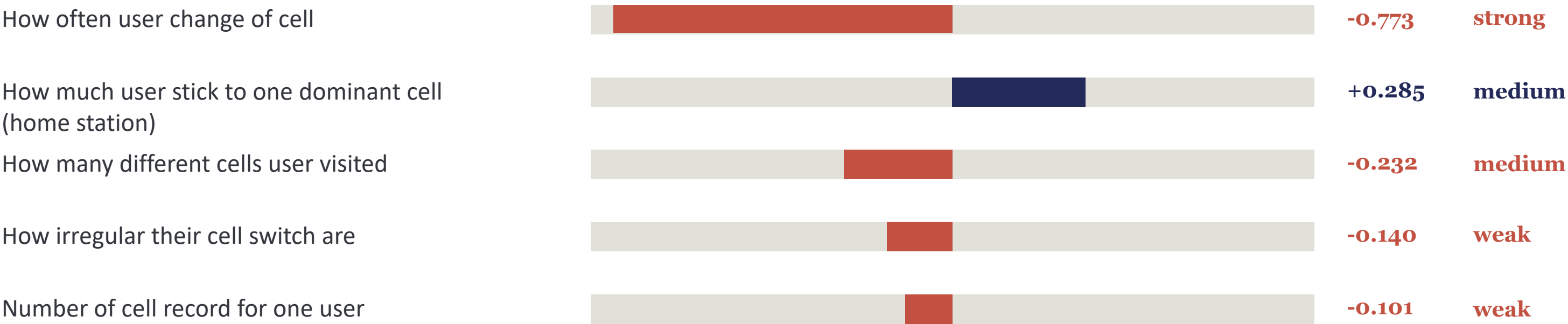


› moving rate: percentage of users who move the most

The behavior of user holds, the volume story doesn't

Pearson correlation between prediction results of *the 100 users from dataset_test* and data of this *dataset_test*

Bar extends right in navy for a positive correlation, left in rust for a negative one — length is proportional to coef.



VERDICT

- › One thing matters, by far: **people who change cell often are much harder to predict.**
- › We should find a way to **improve LLM training** for users who move a lot.

Some tools I can reuse from their work

WILL THEY MOVE, OR STAY?

Tiphain and Benjamin clean a dataset with removing staying user and I could reuse it to train model with this moving of user.



WHAT DID SIMILAR PEOPLE DO?

She also compares our user's trip with other people who made the exact same journey before.



If most of them end up in the same cell, that's a strong clue for our user too.

GOAL Reuse both of these tools in my own model training.

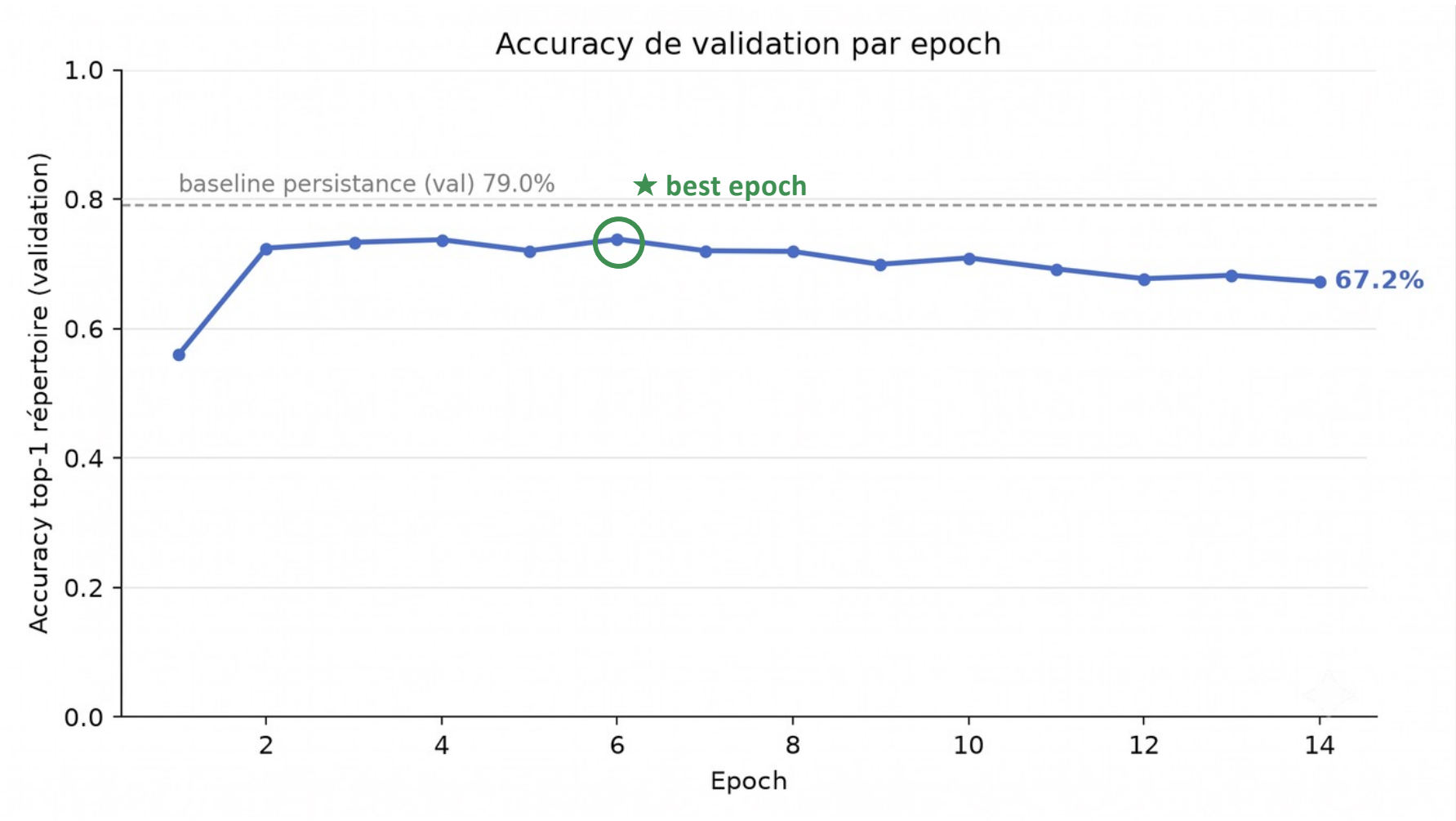
Presentation 4

Rework dataset and add context to try to get better results

Olivier Recher
July 17 - July 21

n_records > 10 & without user that leave the area during the day

Qwen2.5-0.5B QLoRA · dataset filtered · early-stopped at epoch 14/20 (patience 8) · best checkpoint = epoch 6 (73.8% during training session)



Filtred = no records in 4 hours during the activity period

FINAL EVAL — 80% prefix / 20% suffix per sequence on 100 users of test

THE MODEL

80.1%

>
+1.1 pts

PERSISTENCE BASELINE

79.0%

predict the last-seen cell, no model at all

Comparison between different splits

Same Qwen2.5-0.5B QLoRA, same filtered dataset, only context_fraction changes

Split (ctx / test)	Training accuracy	Test accuracy	Baseline persist.	Gap vs baseline	Test top-3	Test top-5	Beat baseline
70% / 30%	72.3%	76.5%	76.8%	-0.3 pts	87.1%	90.1%	✗ No
80% / 20%	73.8%	80.1%	79.0%	+1.1 pts	89.1%	91.6%	✓ Yes
90% / 10%	75.7%	77.7%	80.1%	-2.4 pts	87.2%	90.5%	✗ No

🏆 80/20 is the only split that beats the baseline

ⓘ Early stop on plateau — 70/30: best val at epoch 5 (9 epochs run) · 80/20: best val at epoch 6 (10 epochs run) · 90/10: best val at epoch 4 (8 epochs run). Baseline persistence differs because the split point moves on the same 500 train users: 70%, 80%, 90%.

VERDICT Only 80/20 beats the persistence baseline (+1.1 pts); 70/30 and 90/10 both trail it (−0.3 and −2.4 pts) despite 90/10 reaching the **highest training accuracy** —> more context makes the held-out target easier to memorize in validation, but that gain doesn't transfer to a fair test comparison.

Analyse of worse result <= 12.5%

“preds.” = number of records in 20% of test · “ceiling” = max score possible (cells never seen are excluded)

User	Preds.	Target never seen	Ceiling	Persistence	Model
1049149	3	3/3	0%	0%	0%
1291964	3	3/3	0%	0%	0%
1344347	8	6/8	25%	12%	12.5%
929903	12	3/12	75%	8%	8.3%
1161518	9	1/9	89%	67%	11.1%
828479	2	0/2	100%	50%	0%
1067194	3	0/3	100%	67%	0%

GROUP A — metric floor, not a model failure

- › Target cells never visited before
- › Ceiling is mathematically 0-25%
- › Not a model issue — a metric limitation

Close to random

GROUP B — real behavior shift

- › Ceiling says 75% is learnable
- › Neither the baseline nor the model tracks the spike

Model needs to be train on user with this kind of behavior shift

Focus

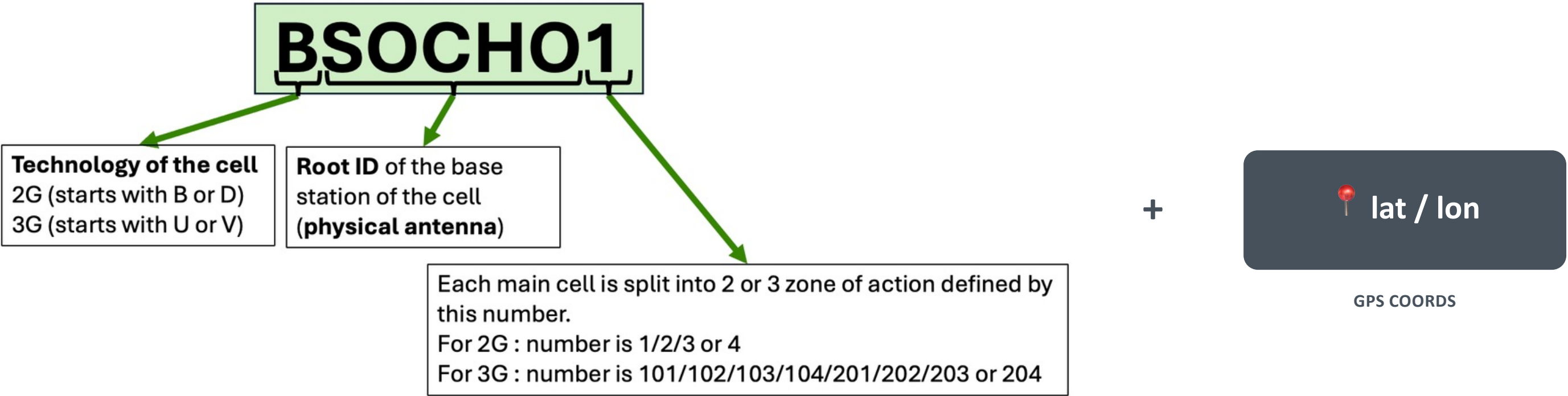
GROUP C — the model loses to a trivial baseline

- › No structural excuse —> ceiling is 89-100%
- › "Repeat the last cell" alone already scores 50-67%
- › Users oscillate between near-twin cells (e.g. UKVCHE102/104)
- › Hypothesis: model confuses the two cells

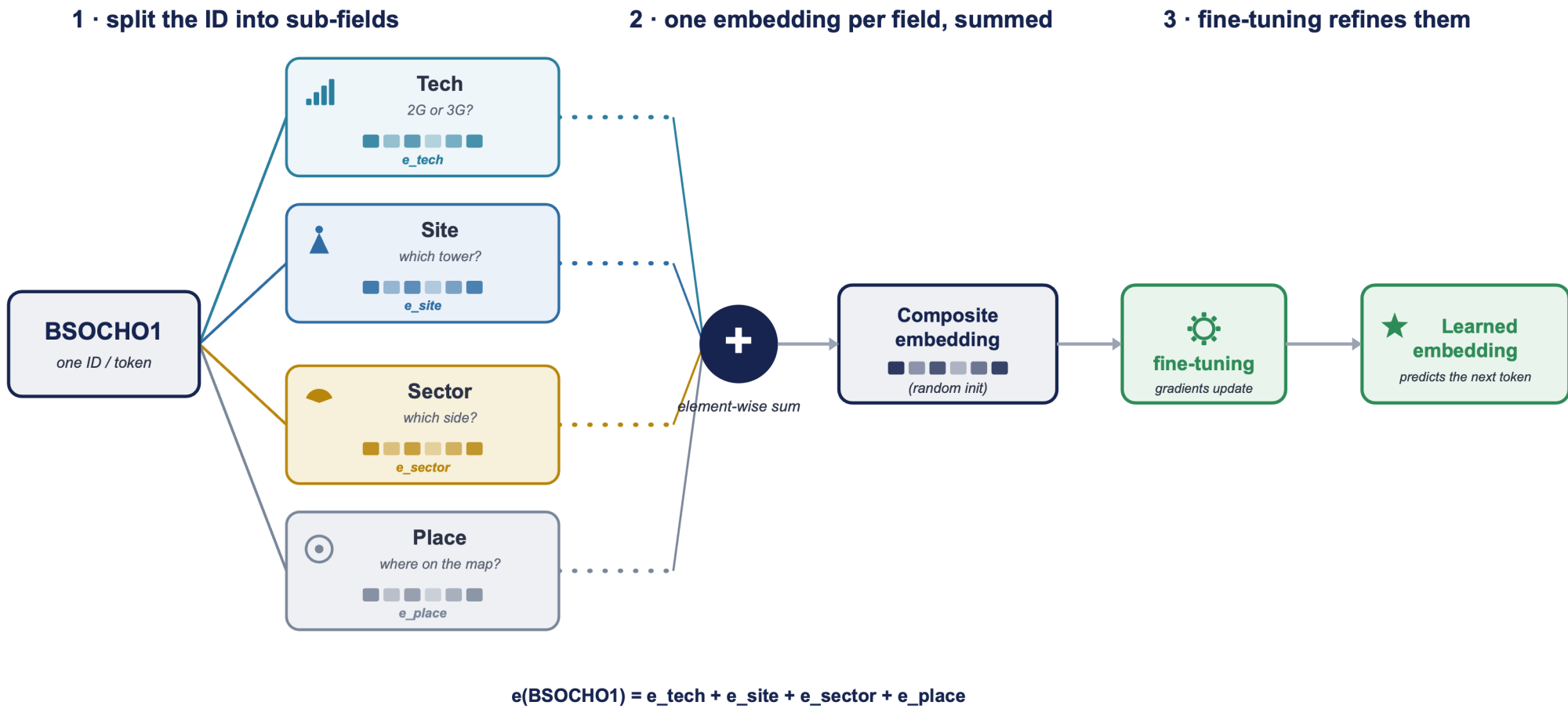
Perhaps by teaching the model **what a cell_id actually is**, we can fix this case.

From an opaque cell_id to structured signal

1 — WHAT WE GIVE THE MODEL



2 — HOW TRAINING USES IT



NEXT STEPS

What comes next

Finish work and apply advise from last time

COMING NEXT Finish the code to teach information relative to a cell_id to the model and run it.

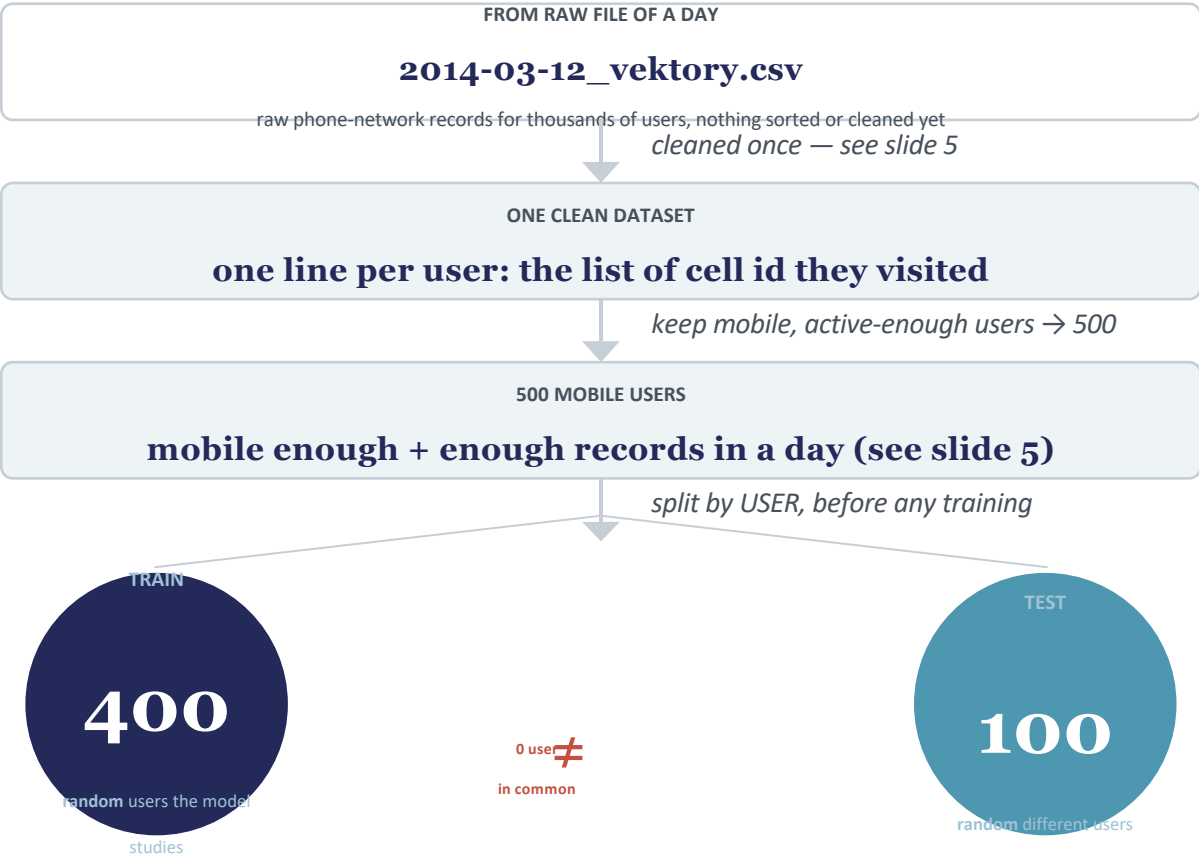
COMING NEXT Use Nino informations about probability and number of visit for each cell applicate to my dataset.

PRESENTATION 5

Rework dataset and add context to try to get better results

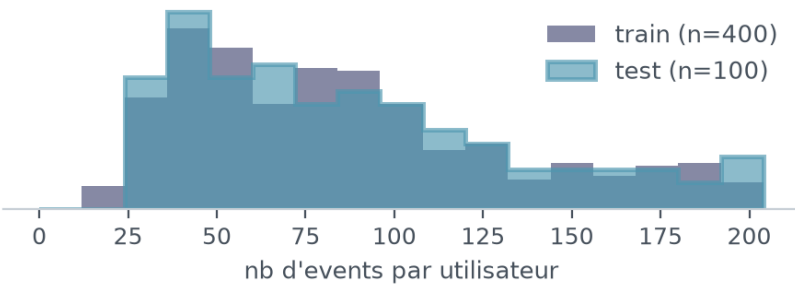
Olivier Recher
July 21 - July 24

500 mobile users (Nino) → split 400 train / 100 test

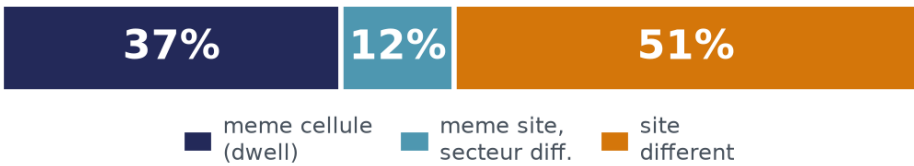


LONGUEUR DES SEQUENCES — train vs test

min 19 · max 200 · moyenne ≈ 86 · train ≈ test, pas de biais



A CHAQUE PAS DE TEMPS, QUE FAIT L'UTILISATEUR ?



TOP 20 CORRIDORS = 15% DES TRAJETS

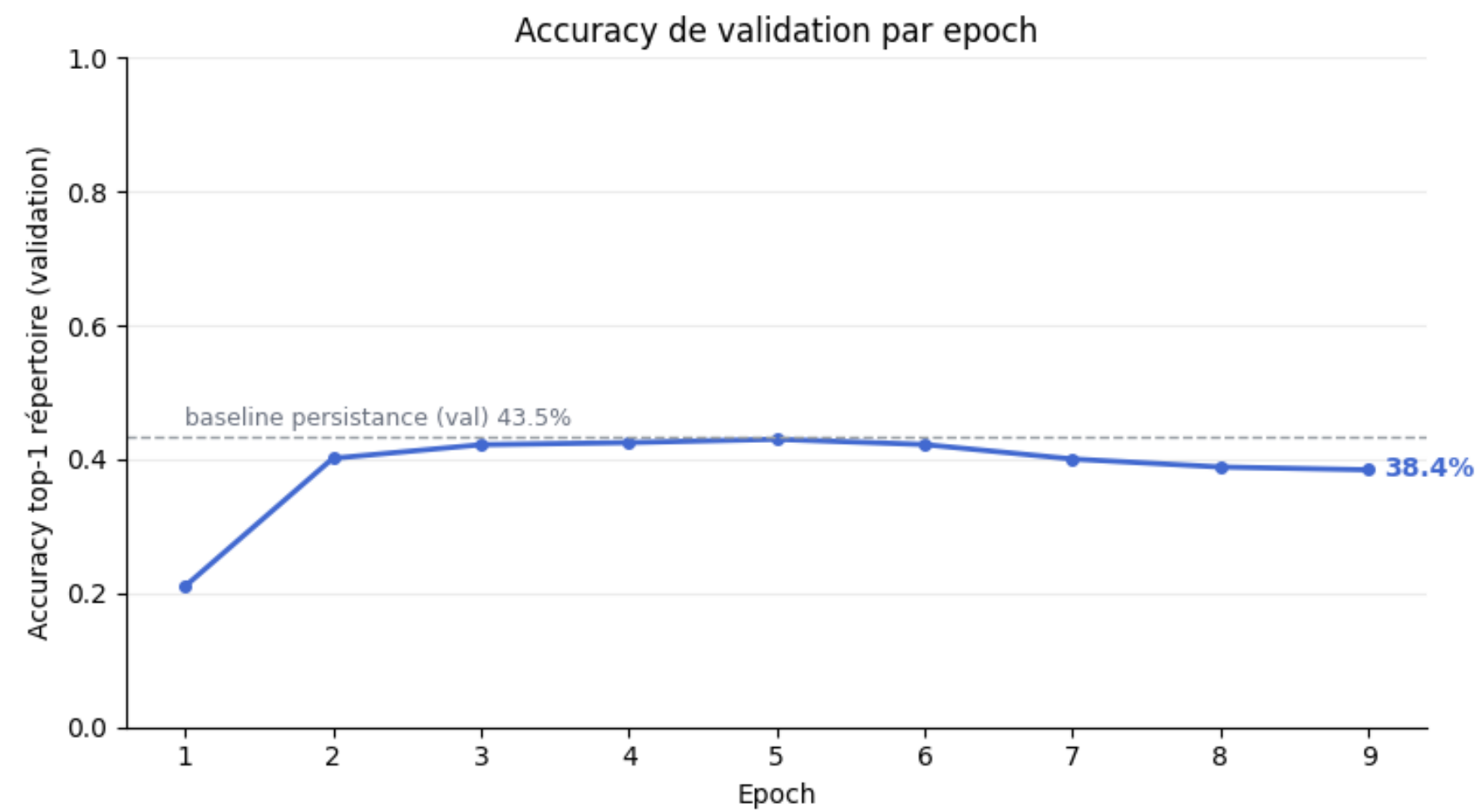
paires reciproques -> aller-retour domicile-travail



Meme comportement de mobilite cote train et cote test : dwelling, handovers, trajets domicile-travail recurrents.

Resultat Nino

Qwen2.5-0.5B QLoRA · dataset mobile users, non-absent (Nino) · best checkpoint = epoch 5 (43.0% during training session)



Mobile users only, excluding
absent users
(dataset provided by Nino)

FINAL EVAL — 80% prefix / 20% suffix per sequence on 100 users of test

THE MODEL

44.6%

<
-0.9 pts

PERSISTENCE BASELINE

45.5%

predict the last-seen cell, no model at all

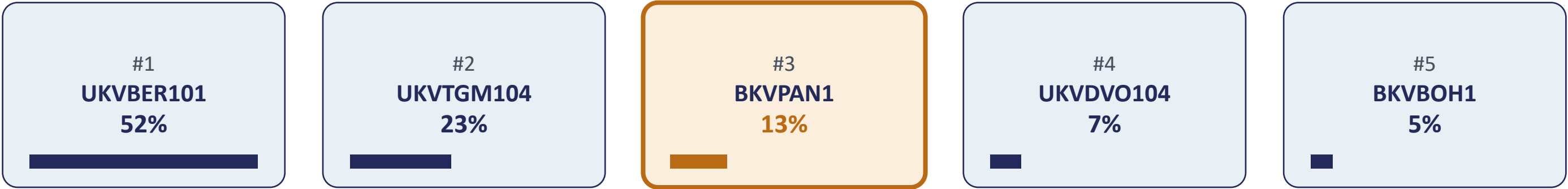
Expliquer le split différent entre train et test

Top-1, Top-3, Top-5 : c'est quoi ?

Le modèle classe ses candidats par ordre de probabilité — le top-N regarde si la bonne réponse figure parmi les N premiers.

Exemple : prédictions du modèle pour la prochaine cellule, triées par probabilité

🎯 cellule réellement visitée



TOP-1	1 candidat considéré	❌ Faux
TOP-3	3 candidats considérés	✅ Vrai
TOP-5	5 candidats considérés	✅ Vrai

💡 Plus la fenêtre s'élargit (top-1 → top-3 → top-5), plus il est facile de compter une prédiction comme correcte — c'est pour ça que le top-3 et le top-5 sont toujours $\geq$ au top-1 dans le tableau qui suit.

Result from different test

On prend le 80/20 pour l'entraînement puis on présente les résultats de différent test, 30/70, 40/60...

Split (ctx / test)	Val. accuracy (train)	Test accuracy	Baseline persist.	Gap vs baseline	Test top-3	Test top-5	Beat baseline
30% / 70%	43.0%	41.5%	42.9%	-0.9 pts	62.1%	69.9%	✗ No
40% / 60%	43.0%	42.0%	43.2%	-0.6 pts	62.1%	69.7%	✗ No
50% / 50%	43.0%	42.0%	43.3%	-0.7 pts	62.0%	69.4%	✗ No
60% / 40%	43.0%	42.7%	44.0%	-0.9 pts	62.1%	69.5%	✗ No
70% / 30%	43.0%	43.6%	44.3%	-0.3 pts	62.1%	69.5%	✗ No
80% / 20% (réf.)	43.0%	44.6%	45.5%	-0.9 pts	63.9%	70.7%	✗ No
90% / 10%	43.0%	47.6%	50.9%	-2.1 pts	68.0%	74.3%	✗ No

! No split beats the baseline on this dataset — closest is 70% context (-0.3 pts)
① Single training on the 80/20 split — early stop on plateau: best val at epoch 5, 9 epochs run (patience 4/20). Every context ratio (30–90%) is the same checkpoint evaluated at inference time on the same 100 held-out test users; baseline persistence differs because the split point moves the last-seen cell used for the naive prediction.

VERDICT No context ratio beats the persistence baseline this run (best gap -0.3 pts at 70% context; worst -2.1 pts at 90%), despite the model improving as context grows and reaching its highest test accuracy at 90% context (47.6%)
—> more context helps the model too, but the naive baseline improves in parallel — on this dataset the model hasn't learned enough yet to close the gap (unlike the filtered-dataset run, which beat baseline at 80/20).

Analyse des erreurs en commun de ces différents tests

“preds.” = number of records in 20% of test · “ceiling” = max score possible (cells never seen are excluded)

User	Preds.	Target never seen	Ceiling	Persistence	Model
1049149	3	3/3	0%	0%	0%
1291964	3	3/3	0%	0%	0%
1344347	8	6/8	25%	12%	12.5%
929903	12	3/12	75%	8%	8.3%
1161518	9	1/9	89%	67%	11.1%
828479	2	0/2	100%	50%	0%
1067194	3	0/3	100%	67%	0%

GROUP A — metric floor, not a model failure

- › Target cells never visited before
- › Ceiling is mathematically 0-25%
- › Not a model issue — a metric limitation

Close to random

GROUP B — real behavior shift

- › Ceiling says 75% is learnable
- › Neither the baseline nor the model tracks the spike

Model needs to be train on user with this kind of behavior shift

Focus

GROUP C — the model loses to a trivial baseline

- › No structural excuse —> ceiling is 89-100%
- › "Repeat the last cell" alone already scores 50-67%
- › Users oscillate between near-twin cells (e.g. UKVCHE102/104)

Perhaps by teaching the model **what a cell_id actually is**, we can fix this case.

What comes next

Conclusion de l'analyse qui donne des avis sur quoi faire pour la prochaine fois dont reprendre le travail de geo

COMING NEXT Finish the code to teach information relative to a cell_id to the model and run it.

COMING NEXT Use Nino informations about probability and number of visit for each cell applicate to my dataset.